Vehicle Routing Problem with Resource-Constrained Pickup and Delivery: A Heuristic-Informed BRKGA with Pattern-Based Analysis

Manbir Sodhi
sodhi@uri.edu
The University of Rhode Island
Kingston, RI, USA

Romesh Satish Prasad
romeshprasad@uri.edu
The University of Rhode Island
Kingston, RI, USA

Abstract

We introduce the Vehicle Routing Problem with Resource-Constrained Pickup and Delivery (VRP-RPD), where agents deploy finite identical resources at customer locations for processing before retrieval and redeployment. Applications include portable medical equipment, tool rental, and disaster relief. Unlike classical pickup-and-delivery variants, VRP-RPD permits different agents to perform dropoff and pickup for the same customer—creating inter-route dependencies absent from standard formulations. We provide a complete mixed-integer linear programming formulation and demonstrate that exact methods are intractable even for small instances. Problems with 16 customers cannot be solved to optimality within two hours of computational time. We develop a Biased Random-Key Genetic Algorithm (BRKGA) with a four-gene-per-customer encoding. Two genes assign dropoff and pickup agents independently, while two priority keys determine sequencing. A simulation decoder guarantees feasibility by deferring operations until resources become available. Experiments on 14 TSPlib-derived benchmarks across five variants of processing time (base, 2X, 5X, 1R10, 1R20) compare four configurations. Warm-start BRKGA achieves 13–67% makespan reduction over the heuristics, with larger gains on higher-resource instances. Ablation tests show warm-start initialization is the primary driver of performance. Friedman tests ($p < 0.01$) confirm warm-start BRKGA superiority across all instance variants.

CCS Concepts

• **Theory of computation** → **Routing and network design problems**; • **Computing methodologies** → **Genetic algorithms**.

Keywords

Vehicle Routing Problem, Pickup and Delivery, BRKGA, Resource Constraints, Makespan Optimization

ACM Reference Format:
Manbir Sodhi and Romesh Satish Prasad. 2026. Vehicle Routing Problem with Resource-Constrained Pickup and Delivery: A Heuristic-Informed BRKGA with Pattern-Based Analysis. In *Genetic and Evolutionary Computation Conference (GECCO '26), July 13–17, 2026, San Jose, Costa Rica.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3795095.3805185

This work is licensed under a Creative Commons Attribution 4.0 International License.
GECCO '26, San Jose, Costa Rica
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2487-9/2026/07
https://doi.org/10.1145/3795095.3805185

1 Introduction

Autonomous mobile robots are increasingly deployed for warehouse inventory scanning, infrastructure inspection, agricultural monitoring, and last-mile delivery. A common operational pattern observed is that robots are transported by vehicles to dispersed service locations, operate independently to complete their tasks, and must subsequently be retrieved. The key insight driving efficiency is that transport vehicles—often the scarcer resource—need not wait while robots work. A delivery vehicle can deploy robots at several locations and continue to additional sites, while a separate vehicle later collects robots from locations where tasks have concluded. This decoupling enables delivery vehicles to maximize coverage during time-critical windows while pickup vehicles optimize routes based on actual completion times.

This operational pattern extends across diverse domains. In disaster response operations, emergency teams deploy drones for aerial damage assessment, portable communication relays, and hazard monitoring sensors to multiple incident sites. With transport vehicles in critically short supply, decoupling delivery and pickup allows deployment teams to maximize initial coverage while separate retrieval teams respond as incidents resolve. Construction equipment rental companies deliver tools—concrete mixers, scaffolding systems, laser levels—to job sites where they operate for specified rental durations before requiring retrieval. Healthcare providers transport portable diagnostic devices to patient homes for prescribed monitoring sessions, with specialized setup crews handling deployment and separate collection vehicles retrieving equipment as procedures complete. Agricultural operations distribute sensors across farmland during narrow weather windows, collecting them on separate schedules as monitoring periods conclude. In each case, resources are dropped off, operate autonomously for some duration, and must eventually be retrieved—potentially by a different vehicle.

The Vehicle Routing Problem (VRP) has generated extensive research across numerous variants [19]. Pickup and delivery problems (PDPs) form a major subclass, classified by Berbeglia et al. [2] into one-to-many-to-one, many-to-many, and one-to-one categories based on origin-destination structure. A fundamental constraint in classical PDPs is that pickup and delivery of each request must be performed by the same vehicle [14]. The PDP with Transfers (PDPT) relaxes this by allowing goods to change

vehicles at designated transshipment locations [12]. However, a common assumption across these variants is that the vehicles remain present at service locations throughout service duration, or service is treated as instantaneous upon arrival.

We introduce the Vehicle Routing Problem with Resource-Constrained Pickup and Delivery (VRP-RPD) to address this gap. In VRP-RPD, vehicles carry limited resources to customer locations, where each resource operates autonomously for a fixed processing time before becoming available for pickup. Critically, different vehicles may perform dropoff and pickup for the same customer at the same location—without requiring intermediate transfer points—enabling flexible coordination and dynamic resource rebalancing. The objective is to minimize makespan—the time when the last vehicle returns to the depot after all resources have been deployed, processed, and retrieved.

Contributions. We formally define VRP-RPD and provide a complete mixed-integer linear programming formulation. We demonstrate computational intractability for instances beyond 16 customers. We develop a BRKGA-based metaheuristic with specialized encoding that exploits the decoupled nature of dropoff and pickup operations. Computational experiments on TSPlib-derived benchmarks [16] demonstrate scalability and quantify the benefits of cross-vehicle coordination compared to same-vehicle formulations.

The paper proceeds as follows. Section 2 reviews related literature and positions VRP-RPD relative to existing problem classes. Section 3 presents the problem formulation including the complete MILP. Section 4 describes our solution methodology. Section 5 presents computational results. Section 6 concludes with practical implications and directions for future research.

2 Literature Review

The VRP-RPD integrates characteristics from vehicle routing, pickup and delivery, and scheduling domains. We review related problem classes, highlighting how VRP-RPD's distinctive features—same-location dropoff-pickup pairs, cross-vehicle coordination, and resource capacity constraints—distinguish it from existing formulations.

2.1 Vehicle Routing Problems

The classical **Vehicle Routing Problem (VRP)**, introduced by Dantzig and Ramser [4], routes a fleet of vehicles to serve customer demands while minimizing travel distance. The **Capacitated VRP (CVRP)** limits total demand each vehicle can serve [19]. While VRP-RPD shares capacity constraints with CVRP, the semantics differ fundamentally—CVRP capacity represents maximum simultaneous load of goods being transported, while VRP-RPD capacity represents resources carried that are deployed at customer locations, decreasing with dropoffs and increasing with pickups. This creates a dynamic capacity profile throughout each route that depends on the interleaving of dropoff and pickup operations.

2.2 Pickup and Delivery Problems

Berbeglia et al. [2] classify **pickup and delivery problems** by request structure: one-to-many-to-one (goods flow between depot and customers), many-to-many (goods flow between arbitrary origin-destination pairs), and one-to-one (each request has a specific origin and destination). A fundamental constraint across all these classes is that pickup and delivery of each request must be performed by the same vehicle [13].

The **VRP with Simultaneous Pickup and Delivery (VRP-SPD)** requires vehicles to handle both operations during a single customer visit [5, 10]. Unlike VRP-RPD, delivery and pickup in VRPSPD occur simultaneously with the vehicle present—there is no autonomous processing period. The **Pickup and Delivery Problem with Time Windows (PDPTW)** transports goods from pickup to delivery locations with time window constraints [6, 17]; it features spatially separated pickup and delivery locations, requires the same vehicle for both operations, and enforces precedence constraints within a single route. In contrast, VRP-RPD features co-located operations (dropoff and pickup occur at the same customer location), permits cross-vehicle coordination (different vehicles may handle dropoff and pickup), and creates precedence dependencies that span multiple routes.

The **Dial-a-Ride Problem (DARP)** routes vehicles for passenger transportation, where passengers board at origins and alight at destinations [3, 14]. DARP requires continuous vehicle presence during transport—passengers cannot be "deployed" to operate independently. VRP-RPD resources, by contrast, function autonomously after deployment. The **PDP with Transfers (PDPT)** allows goods to change vehicles at designated transshipment locations [12]. This introduces cross-vehicle coordination but requires explicit transfer points where vehicles must rendezvous. VRP-RPD dropoff and pickup occur at the same customer location without requiring intermediate transfer points or vehicle synchronization.

2.3 Scheduling and Technician Routing

The **Technician Routing and Scheduling Problem (TRSP)** routes service personnel accounting for skills, tools, and service durations [11, 15]. Unlike VRP-RPD, technicians in TRSP remain on-site throughout service duration—they cannot deploy a resource and depart while service continues autonomously.

The **Traveling Salesman Problem with Jobs (TSPJ)** involves a single traveler visiting nodes to initiate jobs that continue autonomously, with the objective of minimizing makespan [18]. TSPJ shares key characteristics with VRP-RPD—co-located initiation and completion, autonomous processing during the traveler's absence, and makespan-based objectives. However, TSPJ involves only a single traveler, does not model resource retrieval (jobs simply complete without requiring collection), and imposes no capacity constraints on how many jobs can be active simultaneously. VRP-RPD extends the TSPJ concept to multiple vehicles with limited capacity, requiring both dropoff and pickup operations while enabling cross-vehicle coordination.

2.4 Positioning VRP-RPD

Table 1 compares VRP-RPD with related problem classes across three dimensions: whether dropoff and pickup occur at the same or different locations, whether the same or different agents can perform paired operations, and how capacity is interpreted. VRP-RPD occupies a unique position combining same-location dropoff-pickup pairs, cross-vehicle coordination capability, and resource

Table 1: Comparison of VRP-RPD with related problem classes

Problem	Location	pickup-dropoff agent	Capacity
VRP	Single visit	N/A	Transported goods
VRPSPD	Simultaneous	Same	Net load change
PDPTW	Different	Same	Transported goods
DARP	Different	Same	Passengers
TRSP	On-site	Same	Tools/skills
TSPJ	Same	Single	Unlimited
VRP-RPD	**Same**	**Different**	**Deployed resources**

capacity constraints reflecting simultaneous deployment—a configuration not addressed by existing formulations.

3 Problem Formulation

In the Vehicle Routing Problem with Resource-Constrained Pickup and Delivery (VRP-RPD), a fleet of m vehicles operates from a central depot, each carrying at most k identical resources. A set of n customers requires service, where each customer c requires three phases: (1) *dropoff*—a vehicle arrives and deploys one resource; (2) *processing*—the resource operates autonomously for p_c time units; and (3) *pickup*—a vehicle arrives and retrieves the resource. The objective is to minimize the makespan—the time at which all vehicles have returned to the depot after completing all operations.

In the *mixed interleaving* variant, the constraint that dropoff and pickup must be performed by the same vehicle is relaxed. Vehicle a may drop off a resource at customer c, while vehicle $b \neq a$ picks up the resource after processing completes. This flexibility enables greater routing efficiency but introduces cross-vehicle coordination requirements: the pickup vehicle must not arrive before processing completes, regardless of which vehicle performed the dropoff.

3.1 MILP Formulation

We present a mixed-integer linear programming formulation for VRP-RPD. Let $C = \{1, \ldots, n\}$ denote customers, $V = \{1, \ldots, m\}$ denote vehicles, and $N = \{0\} \cup C$ denote all locations including the depot (index 0). Parameters include vehicle capacity k, travel time $d_{i,j}$ from i to j, processing time p_c at customer c, and a sufficiently large constant M.

Decision Variables. Binary routing variables $x_{i,j,v} \in \{0,1\}$ indicate whether vehicle v travels directly from location i to j. Binary visit variables $y_{c,v} \in \{0,1\}$ indicate whether vehicle v visits customer c. Binary operation variables $\delta_{c,v}$ and $\pi_{c,v}$ indicate whether vehicle v performs dropoff or pickup at customer c, respectively. Continuous timing variables include: $t_{i,v}$, the arrival time of vehicle v at location $i \in N$; $t^{\text{dep}}_{i,v}$, the departure time of vehicle v from location $i \in N$; $T_{\text{drop}}[c]$ and $T_{\text{pickup}}[c]$, the actual dropoff and pickup times at customer c; and $t_{\text{return},v}$, the time vehicle v returns to the depot. Capacity variable $q_{i,v}$ denotes the number of resources on vehicle v immediately after completing service at location $i \in N$. Integer variable $u_{c,v}$ denotes the position of customer c in vehicle v's route for subtour elimination. The makespan T is the objective variable to be minimized.

Service Constraints. Each customer receives exactly one dropoff and one pickup, possibly by different vehicles:

$$\sum_{v \in V} \delta_{c,v} = 1, \quad \sum_{v \in V} \pi_{c,v} = 1, \quad \forall c \in C \tag{1}$$

Depot Constraints. Each vehicle departs from and returns to the depot exactly once:

$$\sum_{j \in C} x_{0,j,v} = 1, \quad \sum_{i \in C} x_{i,0,v} = 1, \quad \forall v \in V \tag{2}$$

Time is anchored at the depot:

$$t_{0,v} = 0, \quad t^{\text{dep}}_{0,v} = 0, \quad \forall v \in V \tag{3}$$

Flow Conservation and Visit Linking. For each customer and vehicle, flow balance defines whether the customer is visited:

$$\sum_{i \in N} x_{i,c,v} = y_{c,v}, \quad \sum_{j \in N} x_{c,j,v} = y_{c,v}, \quad \forall c \in C, v \in V \tag{4}$$

Self-loops are prohibited:

$$x_{i,i,v} = 0, \quad \forall i \in N, v \in V \tag{5}$$

A vehicle must visit a customer to perform an operation:

$$\delta_{c,v} \leq y_{c,v}, \quad \pi_{c,v} \leq y_{c,v}, \quad \forall c \in C, v \in V \tag{6}$$

Operation Timing. If vehicle v performs dropoff at c, dropoff occurs at arrival:

$$t_{c,v} - M(1 - \delta_{c,v}) \leq T_{\text{drop}}[c] \leq t_{c,v} + M(1 - \delta_{c,v}), \quad \forall c \in C, v \in V \tag{7}$$

If vehicle v performs pickup at c, pickup occurs at departure:

$$t^{\text{dep}}_{c,v} - M(1 - \pi_{c,v}) \leq T_{\text{pickup}}[c] \leq t^{\text{dep}}_{c,v} + M(1 - \pi_{c,v}), \quad \forall c \in C, v \in V \tag{8}$$

Processing must complete before pickup, enforced globally independent of vehicle identity:

$$T_{\text{pickup}}[c] \geq T_{\text{drop}}[c] + p_c, \quad \forall c \in C \tag{9}$$

This cross-vehicle precedence constraint creates the inter-route dependencies that distinguish VRP-RPD from classical pickup-and-delivery formulations.

Departure Time. Departure is at least arrival:

$$t^{\text{dep}}_{c,v} \geq t_{c,v}, \quad \forall c \in C, v \in V \tag{10}$$

If the same vehicle performs both dropoff and pickup at c, it must wait for processing:

$$t^{\text{dep}}_{c,v} \geq t_{c,v} + p_c(\delta_{c,v} + \pi_{c,v} - 1), \quad \forall c \in C, v \in V \tag{11}$$

Time Propagation. Travel time consistency along routes:

$$t_{j,v} \geq t^{\text{dep}}_{i,v} + d_{i,j} - M(1 - x_{i,j,v}), \quad \forall i, j \in N, v \in V \tag{12}$$

Capacity Constraints. Vehicles depart the depot with full capacity:

$$q_{0,v} = k, \quad \forall v \in V \tag{13}$$

Capacity bounds:

$$0 \leq q_{i,v} \leq k, \quad \forall i \in N, v \in V \tag{14}$$

Capacity propagates along arcs and updates at customers:

$$q_{j,v} \geq q_{i,v} - \delta_{j,v} + \pi_{j,v} - M(1 - x_{i,j,v}), \quad \forall i \in N, j \in C, v \in V \tag{15}$$

$$q_{j,v} \leq q_{i,v} - \delta_{j,v} + \pi_{j,v} + M(1 - x_{i,j,v}), \quad \forall i \in N, j \in C, v \in V \tag{16}$$

Dropoff requires carrying at least one resource:

$$q_{i,v} \geq \delta_{j,v} - M(1 - x_{i,j,v}), \quad \forall i \in N, j \in C, v \in V \tag{17}$$

Pickup cannot exceed capacity:

$$q_{i,v} + \pi_{j,v} \leq k + M(1 - x_{i,j,v}), \quad \forall i \in N, j \in C, v \in V \tag{18}$$

Subtour Elimination. While time propagation with strictly positive travel times implicitly prevents subtours, we include explicit Miller-Tucker-Zemlin (MTZ) constraints to strengthen the LP relaxation:

$$1 \leq u_{c,v} \leq |C|, \quad \forall c \in C, v \in V \tag{19}$$

$$u_{i,v} - u_{j,v} + |C|\, x_{i,j,v} \leq |C| - 1, \quad \forall i, j \in C, i \neq j, v \in V \tag{20}$$

Makespan Objective. Return time is determined by the last arc into the depot:

$$t_{\text{return},v} \geq t^{\text{dep}}_{c,v} + d_{c,0} - M(1 - x_{c,0,v}), \quad \forall c \in C, v \in V \tag{21}$$

The makespan is the maximum return time:

$$T \geq t_{\text{return},v}, \quad \forall v \in V \tag{22}$$

$$\text{Minimize} \quad T \tag{23}$$

3.2 Computational Complexity and Bound Quality

Theorem 3.1. *VRP-RPD is NP-hard.*

Proof. By reduction from the min-max Vehicle Routing Problem (min-max VRP). Consider a VRP-RPD instance where processing times $p_c = 0$ for all customers $c \in C$, and vehicle capacity $k \geq n$. Since processing time is zero, pickup can occur immediately after dropoff at each customer. Since capacity is non-binding, each customer requires exactly one visit where both operations are performed instantaneously by the same vehicle. The problem reduces to partitioning customers among m vehicles and sequencing visits to minimize the maximum return time—precisely the min-max VRP. Since min-max VRP is NP-hard [20], VRP-RPD is NP-hard. □

We validated the MILP formulation on small test instances with 5 customers, where Gurobi 10.0 [9] produced optimal solutions within seconds. However, the formulation exhibits severe scalability limitations due to the extensive use of big-M constraints in time propagation and precedence linking, which yield weak linear programming relaxations. For instances such as `gr17` (17 customers) and `gr24` (24 customers), Gurobi obtained feasible solutions but with optimality gaps of approximately 70% after two hour of computation on a workstation with 32 CPU cores and 128GB RAM—indicating the solver found valid solutions but could not substantially tighten the lower bound. For larger instances (50+ customers), the number of binary variables grows as $O(n^2 m)$ and continuous variables as $O(nm)$, exceeding practical memory limits before meaningful bound improvement occurred. Consequently, we do not report optimality gaps for metaheuristic solutions, as the LP relaxation bounds are too weak to provide meaningful quality certificates. This intractability motivates our metaheuristic approach, which produces high-quality solutions for instances with hundreds of customers within minutes. Developing tighter formulations or valid inequalities for VRP-RPD remains an open direction for future research.

4 Solution Methodology

We develop a Biased Random-Key Genetic Algorithm (BRKGA) tailored to VRP-RPD's structure. BRKGA's random-key encoding [1, 8] naturally handles the assignment and sequencing decisions in VRP-RPD, while a simulation-based decoder manages the inter-route dependencies arising from cross-vehicle coordination. Section 6 presents ablation experiments quantifying the contribution of each component.

Chromosome Encoding. VRP-RPD requires representing both agent assignments and operation sequencing, with independent encoding of dropoff and pickup operations. Each customer $i \in \{1, \ldots, n\}$ is represented by four genes: $\text{Gene}_{i,0}, \text{Gene}_{i,1} \in [0, 1]$ for dropoff/pickup agent assignment, and $\text{Gene}_{i,2}, \text{Gene}_{i,3} \in [0, 1]$ for dropoff/pickup priority keys. The complete chromosome is $\mathbf{c} \in [0, 1]^{4n}$. Agent assignment genes map to discrete vehicle identifiers via $\text{agent\_id} = \lfloor \text{Gene} \times m \rfloor$, where m is the number of vehicles. When $\text{Gene}_{i,0}$ and $\text{Gene}_{i,1}$ map to different agents, customer i's operations are performed by different vehicles, enabling cross-vehicle coordination.

Simulation-Based Decoder. The decoder transforms chromosomes into feasible solutions via discrete-event simulation. Each agent a maintains state (ℓ_a, τ_a, r_a): current location, current time, and number of resources currently carried. From the chromosome, the decoder constructs event lists sorted by priority keys, then processes events using greedy selection of the next feasible operation. A dropoff is feasible if $r_a > 0$ (agent has resources to deploy). A pickup of customer i by agent a is feasible if: (1) dropoff has already occurred at time τ_i^D, (2) arrival time satisfies $\tau_a + d(\ell_a, \ell_i) \geq \tau_i^D + p_i$ (processing is complete), and (3) $r_a < k$ (capacity available for retrieved resource). When no feasible events exist but customers remain unserviced, a rescue mechanism assigns pickups to the nearest available agent, overriding chromosome assignments. This guarantees feasibility regardless of chromosome quality, ensuring the genetic algorithm never encounters infeasible solutions. The makespan is $f(\mathbf{c}) = \max_a(\tau_a + d(\ell_a, 0))$ with decoder complexity $O(nm + n \log n)$.

Evolutionary Operators. The population of size P is partitioned into elite ($p_e = 0.20$), non-elite, and mutant ($p_m = 0.10$) sets. Offspring are generated via biased crossover: an elite parent $\mathbf{p}_1$ and non-elite parent $\mathbf{p}_2$ produce offspring where each gene inherits from $\mathbf{p}_1$ with probability $\rho_e = 0.70$. Two mutation operators are applied with probability p_{mut}: agent reassignment replaces a random agent gene with $\text{Uniform}(0, 1)$; priority perturbation adds $\mathcal{N}(0, \sigma^2)$ noise to priority keys.

Parallel Island Model. We employ K islands evolving in parallel with heterogeneous parameters to maintain population diversity. GPU islands use population size $P = 512$ with mutation rates distributed over $[0.10, 0.28]$; CPU islands use $P = 256$ with rates

over $[0.15, 0.33]$. Islands operate asynchronously, reporting best solutions to a shared queue without synchronization barriers.

Construction Heuristics. Three heuristics generate initial solutions for warm-start and serve as baselines: (1) *Nearest neighbor* iteratively selects the closest feasible operation (dropoff or pickup) from the current location; (2) *Greedy-defer* applies a penalty multiplier λ to pickup operations, biasing selection toward completing dropoffs before retrievals, with a list-based processor selecting the minimum-cost feasible operation at each step; (3) *Max-regret insertion* computes for each unassigned customer the difference between its best and second-best insertion cost across all routes, prioritizing customers with highest regret to avoid poor placements.

Warm Start Initialization. Heuristic solutions seed the initial population to accelerate convergence. Solutions are converted to chromosomes by encoding agent assignments as $\text{Gene}_{i,j} = (a_i^j + 0.5)/m$ and priority keys as normalized tour positions, ensuring decoder fidelity: $\mathcal{D}(\text{encode}(S)) = S$. The top 20% of the initial population comprises heuristic solutions; the remainder are random chromosomes.

Genetic Programming Hyper-Heuristic. A GP layer extracts patterns from elite solutions and generates candidates that exploit learned structure. Every G_{cycle} generations, the analyzer extracts building blocks (high-frequency, low-variance gene subsequences) and generates candidates via three strategies: block recombination combines proven partial solutions from different elites; interpolation/extrapolation generates convex combinations of elite chromosomes following BLX-α crossover [7], with extrapolation permitted to explore beyond parental bounds; guided mutation perturbs elite solutions using Gaussian noise. Candidates are injected into islands replacing worst non-elite individuals—we term this process *gene injection*. A diversity filter rejects candidates too similar to existing elites.

5 Experimental Setup

Benchmark Instances We evaluate VRP-RPD on 14 benchmark instances derived from TSPlib [16], using edge weights as travel times. Customer processing times are sampled uniformly from $[d_{\min}, d_{\max}]$, where $d_{\min}$ and $d_{\max}$ are the minimum and maximum edge weights in each instance, defining the *base* variant. Four additional variants modify processing times: **2**× and **5**× apply fixed multipliers, while **1R10** and **1R20** multiply each customer's processing time by a random integer in $[1, 10]$ and $[1, 20]$, respectively. We generate one instance per dataset for the base, 2×, and 5× variants, and 10 instances per dataset for each heterogeneous variant, yielding 322 total instances (14 datasets × 23). For instances with fewer than 24 customers, we use $m = 3$ vehicles with capacity $k = 5$; otherwise, $m = 6$ vehicles with capacity $k = 4$. These variants are designed to test the hypothesis that coordination opportunities increase with processing duration: longer processing times allow vehicles to serve additional customers while resources operate autonomously. Datasets,solvers and results could be accessed from the git repository.

Processing-Time Trend Validation. The processing-time variants test the hypothesis that cross-vehicle coordination benefits increase with processing duration. When processing times are short relative to travel times, vehicles gain little from leaving resources and returning later; as processing times grow, resources operate autonomously for longer, enabling vehicles to serve additional customers. The multiplied variants (2×, 5×) amplify these opportunities, while the heterogeneous variants (1R10, 1R20) assess robustness under variability.

We validate this assumption using seven benchmark datasets and three processing-time variants (base, 1R10, 1R20). Per-dataset rank-based trend analysis reveals a consistent monotonic increase in interleaving benefits as processing times rise (mean Spearman $\rho = 0.43$), with positive trends observed in 6 of 7 datasets. This effect is supported by complementary parametric and non-parametric tests and confirmed by an omnibus Friedman test (Table 2). Together, these results demonstrate that coordination benefits grow systematically with processing duration, supporting the core hypothesis.

Algorithm Configurations We compare five configurations to isolate the contribution of each component:

(1) **Heur**: Best of nearest neighbor, greedy-defer, and max-regret heuristics.
(2) **BRKGA**: Random initialization, no gene injection.
(3) **CSGI**: Random initialization with GP-driven gene injection during evolution.
(4) **WS**: Heuristic-seeded initialization, no gene injection.
(5) **WSGI**: Heuristic-seeded initialization with GP-driven gene injection.

All BRKGA variants share the same encoding, decoder, operators, and island model; they differ only in initialization and use of gene injection.

Table 2: Statistical Analysis of Interleaving Benefits Across Processing Time Variants

Trend Analysis (Primary Result)
Per-dataset Spearman correlation: Mean $\rho = 0.429$ (SD = 0.450)
One-sample t-test: $t = 2.52$, $p = 0.023$
Wilcoxon signed-rank: $W = 24.5$, $p = 0.055$
Friedman test (omnibus): $\chi^2(2) = 6.00$, $p = 0.050$
Datasets with positive trend: 6/7 (86%)

Computational Environments Experiments run on four NVIDIA L40S GPUs with asynchronous island execution. Each run terminates after 5000 generations with 512 population. Identical random seeds are used across configurations for each instance. Performance is measured by makespan, and statistical comparisons are reported in Section 6.

6 Results

This section reports percentage improvements in makespan relative to the heuristics-only baseline (Heur). We compare four BRKGA configurations: base BRKGA with random initialization (BRKGA), cold-start with gene injection during evolution (CSGI), warm-start without gene injection (WS), and warm-start with gene injection during evolution (WSGI). Gene injection operates periodically throughout the evolutionary run, extracting building blocks from elite solutions and injecting new candidates into the population every G_{cycle} generations—it is distinct from the

initialization strategy. Results are presented separately for each processing-time variant.

6.1 Base Variant (Table 3)

. Warm-start variants consistently outperform both Heur and pure BRKGA, with WS achieving the best solution 6 times across 14 datasets. Improvements are particularly notable for mid-sized instances like `gr202` and `gr431`. However, on very large instances (`dsj1000`, `gr666`, `rat783`), CSGI struggles to beat heuristics due to low population size.

6.2 2x and 5x Processing Times (Tables 4 , 5)

As processing times increase, performance differences become more pronounced. All BRKGA methods substantially outperform Heur, validating that metaheuristic search becomes increasingly valuable as coordination opportunities expand. For the 2× variant, WS achieves the best rank, with large instances consistently showing improvements exceeding 40%. In the 5× variant, gains are substantial across all datasets (34–60%). The wins are relatively evenly distributed across BRKGA variants at higher processing scales.

6.3 Randomized Variants (Tables 6, 7)

Stochastic processing time multipliers test algorithm robustness to variable problem structure. All BRKGA methods substantially outperform Heur, achieving 54–67% improvements on large instances. For 1R10, CSGI achieves the best rank, while for 1R20, WS leads. Pure BRKGA achieves the most wins (6) on 1R20, suggesting greater robustness to high stochastic variation.

6.4 Impact of Population Size and ALNS Warm-Start

Table 8 examines two extensions on the base variant: (1) increasing population size from 512 to 10,000 with 2,000 generations, and (2) combining this larger population with ALNS (Adaptive Large Neighborhood Search) based warm-start initialization. ALNS is used as a warm-start because it produces structurally diverse, high-quality solutions that are difficult to obtain from simple constructive heuristics, providing a strong starting population for evolutionary search. Increasing population size (WS 10000) yields substantial improvements over the baseline WS configuration, with gains ranging from 2.39% (`gr17`) to 47.49% (`eil101`). Large instances show particularly strong performance: `dsj1000` achieves 21.99% improvement, `gr666` achieves 40.82%, and `kroA100` achieves 44.39%. Combining the larger population with ALNS initialization (WS + ALNS 10000) produces the best results in 10 of 14 instances, with the strongest improvements on `gr666` (44.73%), `kroA100` (46.35%), and `eil101` (46.91%).

However, ALNS as initialization shows slight negative performance on `gr17` compared to WS. These results demonstrate that larger populations enable more effective exploration of the solution space and that ALNS can provide high-quality initial solutions when paired with sufficient evolutionary search. The computational cost (10,000 population and 2,000 generations) limited evaluation to the base variant only, but this finding suggests population size scaling on GPU architectures represents a promising direction for further improvement.

Table 3: Mean % improvement over Heur: Base variant.

Dataset	Heur	BRKGA	CSGI	WSGI	WS
bays29	1704.00	**20.77**	18.08	14.03	17.55
berlin52	9076.00	**24.16**	17.57	15.92	18.25
dsj1000	118271669	-5.87	-5.11	30.15	**31.39**
eil101	918.00	**20.15**	8.93	8.93	6.64
eil51	420.00	10.00	**14.29**	7.38	13.57
gr17	2738.00	26.77	29.29	43.35	**43.50**
gr202	118892.00	**38.89**	33.53	38.62	37.39
gr21	2792.00	-1.04	**4.19**	**4.19**	0.68
gr24	2283.00	28.25	27.86	26.72	**37.98**
gr431	769903.00	25.03	22.86	**43.17**	30.17
gr48	5741.00	17.85	14.79	16.46	**24.26**
gr666	1170180	-0.12	-0.47	12.57	**14.74**
kroA100	39484.00	**22.07**	11.33	6.54	4.43
rat783	41098.00	1.14	-1.21	4.70	**5.02**
Wins	-	5	2	2	6

Table 4: Mean % improvement over Heur: 2× variant.

Dataset	Heur	BRKGA	CSGI	WSGI	WS
bays29	2438.00	**25.68**	18.42	19.48	21.66
berlin52	15283.00	34.12	33.14	**43.28**	35.38
dsj1000	203504762	28.23	28.51	53.69	**54.92**
eil101	1587.00	37.68	**41.78**	39.45	36.93
eil51	727.00	32.32	**37.00**	34.53	34.53
gr17	3680.00	24.27	38.48	**49.95**	47.39
gr202	197321.00	**52.58**	52.57	51.10	51.92
gr21	4028.00	2.18	3.10	**22.94**	16.51
gr24	2823.00	**39.25**	33.37	34.86	29.05
gr431	1307665	46.89	45.75	46.63	**48.40**
gr48	8984.00	31.90	**32.25**	26.41	24.93
gr666	2256865	40.74	40.03	40.87	**42.11**
kroA100	66803.00	39.69	37.38	42.10	**43.81**
rat783	72681.00	34.74	34.21	52.61	**53.37**
Wins	-	3	3	3	5

6.5 Statistical Validation

Because absolute makespan values vary across datasets and variants, all comparisons rely on paired non-parametric tests assessing relative performance within each instance. All methods were evaluated on identical instances with fixed random seeds.

Table 9 reports Friedman test results. For every variant, the null hypothesis is rejected at $\alpha = 0.05$, confirming that algorithm choice has a statistically significant effect on solution quality.

Average ranks are shown in Table 10. Heur is consistently the worst-performing method, underscoring the importance of metaheuristic search. Among BRKGA variants, WS attains the best or near-best average rank in most variants, while WSGI and CSGI lead in specific settings (1R20 and 1R10, respectively).

Table 5: Mean % improvement over Heur: 5× variant.

Dataset	Heur	BRKGA	CSGI	WSGI	WS
bays29	4625.00	**28.09**	22.25	26.92	25.36
berlin52	26789.00	29.47	**36.09**	21.19	19.48
dsj1000	429802387	47.84	**49.26**	48.88	48.44
eil101	3449.00	51.09	52.33	**52.59**	50.45
eil51	1226.00	28.22	**32.14**	25.94	28.22
gr17	5816.00	**34.75**	25.91	32.44	32.74
gr202	425348.00	**58.64**	58.24	57.29	57.65
gr21	7736.00	30.80	36.17	33.48	**36.56**
gr24	5709.00	48.45	39.90	47.33	**48.47**
gr431	2676956	55.76	**57.41**	56.07	56.19
gr48	15747.00	**39.56**	34.78	31.64	36.06
gr666	4938132	59.09	60.19	**61.20**	59.57
kroA100	140206.00	51.07	52.00	**52.47**	50.20
rat783	150034.00	52.05	52.54	51.82	**52.58**
Wins	-	4	4	3	3

Table 6: Mean % improvement over Heur: 1R10 variant.

Dataset	Heur	BRKGA	CSGI	WSGI	WS
bays29	5907.70	**26.31**	26.01	25.58	25.27
berlin52	36605.80	48.85	**49.09**	44.92	47.26
dsj1000	552124204	58.46	59.26	**59.60**	58.76
eil101	3990.10	53.39	54.32	54.50	**54.68**
eil51	1617.20	40.54	41.30	39.47	**44.15**
gr17	6733.00	19.21	19.59	20.19	**20.21**
gr202	498332.90	**60.06**	59.98	59.82	59.75
gr21	10138.00	36.38	34.52	35.42	**37.76**
gr24	6866.30	45.33	**45.86**	45.58	**45.86**
gr431	3510529	**64.34**	63.91	63.99	64.26
gr48	20160.40	**45.41**	42.04	39.96	43.77
gr666	6010575	65.69	65.62	65.46	**65.73**
kroA100	160956.90	52.61	**53.94**	52.40	53.93
rat783	192744.70	**61.69**	61.22	60.85	61.1
Wins	-	5	3	1	6

Effect of Gene Injection. Table 11 isolates the effect of gene injection by comparing otherwise identical configurations. For cold starts, gene injection yields mixed results, with modest improvements in some variants and degradations in others. In contrast, for warm starts, gene injection generally improves performance. This suggests that gene injection is most effective when combined with warm-start initialization.

Post-hoc Analysis. Nemenyi post-hoc tests using ranks from Table 10 yield a critical difference of 1.63 for $k = 5$ algorithms and $n = 14$ instances at $\alpha = 0.05$ (1.69 for $n = 13$ on 1R10, 2.16 for $n = 11$ on 1R20). Across all variants, all BRKGA configurations significantly outperform Heur (rank differences 1.43–3.00). Among BRKGA variants, WSGI and WS consistently achieve the lowest ranks, though differences among BRKGA configurations generally do not exceed the critical difference under Nemenyi correction. The consistent patterns in Table 11 provide complementary evidence on gene injection effects, suggesting its benefit is context-dependent rather than universal.

Table 7: Mean % improvement over Heur: 1R20 variant.

Dataset	Heur	BRKGA	CSGI	WSGI	WS
bays29	9957.00	**17.76**	11.46	16.53	14.83
berlin52	64699.80	45.95	42.87	46.08	**46.20**
dsj1000	1038842225	63.34	62.95	**63.52**	52.92
eil101	7569.90	**59.98**	59.14	57.66	57.30
eil51	2890.30	**46.55**	45.58	31.11	40.64
gr17	11615.80	10.23	-9.31	**17.59**	16.12
gr202	956988.90	62.82	**63.76**	61.73	57.54
gr21	17817.20	26.14	26.14	26.14	**33.35**
gr24	11933.40	38.26	41.06	41.40	**42.41**
gr431	6671498	67.20	**68.64**	66.85	61.40
gr48	34978.70	39.14	43.67	**43.83**	40.08
gr666	10616970	**68.05**	66.77	67.19	59.00
kroA100	296875.60	**57.16**	55.17	55.32	54.03
rat783	362715.20	**66.86**	64.33	64.54	55.92
Wins	-	6	2	3	3

Table 8: Impact of ALNS warm-start and increased population on Base variant performance.

Dataset	WS (pop:512)	ALNS (%)	WS pop:10000 (%)	WS + ALNS pop:10000 (%)	Best Makespan
bays29	1,405	-20.78	14.52	**19.36**	1,133
berlin52	7,221	-22.93	27.79	**28.40**	5,170
dsj1000	81,151,988	-45.74	21.99	**26.88**	59,335,133
eil101	857	-7.12	**47.49**	46.91	450
eil51	363	-15.70	**32.78**	33.33	242
gr17	1,547	-27.86	**2.39**	-0.32	1,510
gr202	74,434	-58.97	**32.91**	31.13	49,940
gr21	2,773	-0.69	18.36	**20.19**	2,213
gr24	1,416	-48.31	**14.34**	13.63	1,213
gr431	537,592	-42.82	29.75	**30.15**	375,505
gr48	4,348	-26.72	20.42	**24.84**	3,268
gr666	997,690	-17.29	40.82	**44.73**	551,423
kroA100	37,333	-3.09	44.39	**46.35**	20,029
rat783	39,035	-5.29	30.67	**35.66**	25,115

Table 9: Friedman test results for each variant.

Variant	Instances	Friedman χ^2	p-value
Base	14	20.66	3.7×10^{-4}
2×	14	30.55	4.0×10^{-6}
5×	14	28.83	8.0×10^{-6}
1R10	14	28.74	9.0×10^{-6}
1R20	14	15.97	3.1×10^{-3}

6.6 Summary

The ablation study yields several key findings. First, metaheuristic search is essential: Heur consistently underperforms all BRKGA-based methods, with makespan reductions of 15–67% depending

Table 10: Average algorithm ranks (1=best, 5=worst).

Variant	Heur	BRKGA	CSGI	WSGI	WS
Base	4.57	2.57	3.14	2.64	**2.07**
2×	5.00	2.71	2.93	2.18	**2.18**
5×	5.00	2.54	2.22	2.71	**2.54**
1R10	5.00	2.46	**2.08**	3.08	2.38
1R20	4.88	2.00	3.06	**2.56**	2.50

Table 11: Effect of gene injection. Δ Rank = (no injection) − (with injection); positive = injection helps.

	Cold Start		Warm Start	
Variant	Δ Rank	p	Δ Rank	p
Base	−0.57	0.104	+0.57	0.049
2×	−0.22	0.391	+0.00	1.000
5×	+0.32	0.217	−0.17	0.858
1R10	+0.38	0.626	+0.70	0.391
1R20	+1.06	0.054	−0.06	0.735

on instance characteristics. The performance gap widens with increasing processing times, confirming that cross-vehicle coordination opportunities require sophisticated search to exploit effectively.

Second, warm-start initialization is the primary driver of performance gains. WS achieves the best average rank in three of five variants without requiring the computational overhead of gene injection. The heuristic solutions provide the evolutionary search with a strong starting point that guides exploration toward promising regions of the solution space.

Third, gene injection exhibits an interesting interaction with initialization strategy. It benefits cold start configurations (CSGI consistently outranks pure BRKGA) by introducing structure into an initially random population. However, it provides no measurable improvement—and sometimes degrades performance—when combined with warm start. This suggests the injected building blocks may disrupt the coherent solution structures inherited from heuristic initialization, effectively adding noise rather than useful guidance.

Fourth, the benefits of cross-vehicle coordination scale with processing duration. Base variants show modest improvements (15–35% typical), while 5× and randomized variants achieve gains exceeding 50–60% on large instances. This validates the core hypothesis motivating VRP-RPD: when resources operate autonomously for extended periods, decoupling dropoff and pickup across different vehicles enables substantial efficiency improvements.

Across the 322 dataset–variant combinations, warm-start methods (WS or WSGI) achieved the best performance in 40 cases (57%), CSGI in 20 cases (29%), and pure BRKGA in 10 cases (14%). Based on these findings, we recommend WS as the preferred configuration as it combines strong performance with algorithmic simplicity and lower computational overhead.

7 Conclusion

This paper introduced the Vehicle Routing Problem with Resource-Constrained Pickup and Delivery (VRP-RPD), a routing variant where autonomous resources are deployed to customer locations, operate independently during a processing period, and are later retrieved—potentially by different vehicles. This decoupling of dropoff and pickup operations distinguishes VRP-RPD from classical pickup and delivery problems and creates opportunities for cross-vehicle coordination that reduce makespan. We provided a complete mixed-integer linear programming formulation and demonstrated that exact methods face severe scalability limitations: while the formulation produces optimal solutions for instances with 5 customers, problems with 16 customers cannot be solved to optimality within one hour. This intractability motivated our metaheuristic approach.

We developed a BRKGA-based metaheuristic with a four-gene encoding that naturally represents the dropoff/pickup decoupling, a simulation-based decoder with rescue mechanisms guaranteeing feasibility, and a parallel island model exploiting GPU acceleration. Construction heuristics provide warm-start initialization, seeding the initial population with high-quality solutions. Experiments on 322 benchmark instances derived from TSPlib demonstrate substantial improvements over heuristic baselines, with makespan reductions of 15–67% depending on instance size and processing time characteristics. As hypothesized, the benefits of cross-vehicle coordination increase with processing duration: base variants show modest gains while 5× and randomized variants exhibit improvements exceeding 50% on large instances.

The ablation study reveals that warm-start initialization is the key algorithmic component. Warm-start BRKGA (WS) achieves the best average rank in four of five instance variants, outperforming both pure BRKGA and configurations augmented with gene injection during evolution. Gene injection benefits cold start but provides no additional improvement when high-quality initial solutions are available—the computational overhead is not justified. We therefore recommend warm-start BRKGA as the preferred approach for VRP-RPD as it delivers strong performance with minimal algorithmic complexity.

Several directions merit future investigation. Exact methods or stronger bounds would help quantify solution quality gaps and validate metaheuristic performance. Heterogeneous fleets with varying capacities and speeds introduce additional modeling complexity relevant to practical applications. Dynamic variants where customer requests arrive online and processing times are uncertain would increase practical relevance for real-time operations. Finally, the VRP-RPD framework naturally extends to multi-depot settings and problems with resource compatibility constraints, where specific resources may be required at specific customer locations.

References

[1] J.C. Bean. Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2):154–160, 1994.
[2] G. Berbeglia, J.F. Cordeau, I. Gribkovskaia, and G. Laporte. Static pickup and delivery problems: A classification scheme and survey. *TOP*, 15(1):1–31, 2007.
[3] J.F. Cordeau and G. Laporte. The dial-a-ride problem: Models and algorithms. *Annals of Operations Research*, 153(1):29–46, 2007.
[4] G.B. Dantzig and J.H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.

[5] J. Dethloff. Vehicle routing and reverse logistics: The vehicle routing problem with simultaneous delivery and pick-up. *OR-Spektrum*, 23(1):79–96, 2001.
[6] Y. Dumas, J. Desrosiers, and F. Soumis. The pickup and delivery problem with time windows. *European Journal of Operational Research*, 54(1):7–22, 1991.
[7] L.J. Eshelman and J.D. Schaffer. Real-coded genetic algorithms and interval-schemata. In *Foundations of Genetic Algorithms 2*, pages 187–202. Morgan Kaufmann, 1993.
[8] J.F. Gonçalves and M.G.C. Resende. Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, 17(5):487–525, 2011.
[9] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, Version 10.0, 2023. https://www.gurobi.com
[10] Ç. Koç, G. Laporte, and M. Türkay. A review of vehicle routing with simultaneous pickup and delivery. *Computers & Operations Research*, 122:104987, 2020.
[11] A.A. Kovacs, S.N. Parragh, K.F. Doerner, and R.F. Hartl. Adaptive large neighborhood search for service technician routing and scheduling problems. *Journal of Scheduling*, 15(5):579–600, 2012.
[12] R. Masson, F. Lehuedé, and O. Péton. An adaptive large neighborhood search for the pickup and delivery problem with transfers. *Transportation Science*, 47(3):344–355, 2013.
[13] S.N. Parragh, K.F. Doerner, and R.F. Hartl. A survey on pickup and delivery problems. Part II: Transportation between pickup and delivery locations. *Journal für Betriebswirtschaft*, 58(2):81–117, 2008.
[14] S.N. Parragh, K.F. Doerner, and R.F. Hartl. Variable neighborhood search for the dial-a-ride problem. *Computers & Operations Research*, 37(6):1129–1138, 2010.
[15] V. Pillac, C. Guéret, and A.L. Medaglia. A parallel matheuristic for the technician routing and scheduling problem. *Optimization Letters*, 7(7):1525–1535, 2013.
[16] G. Reinelt. TSPLIB—A traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.
[17] S. Ropke and D. Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.
[18] M. Mosayebi, M. Sodhi, and T. A. Wettergren, "The traveling salesman problem with job-times (TSPJ)," *Computers & Operations Research*, vol. 129, p. 105226, 2021.
[19] P. Toth and D. Vigo. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, 2nd edition, 2014.
[20] H. W. Lenstra, Jr., Integer programming with a fixed number of variables, *Mathematics of Operations Research*, vol. 8, no. 4, pp. 538–548, 1983.